

Exemplo de mapeamento

```
"Mappings" : {  
  "AWSRegionToAMI" : {  
    "us-east-1" : { "AMI" : "ami-76817c1e" },  
    "us-west-2" : { "AMI" : "ami-d13845e1" },  
    "us-west-1" : { "AMI" : "ami-f0d3d4b5" },  
    "eu-west-1" : { "AMI" : "ami-892fe1fe" }, ...  
  }  
}
```

```
"ImageId" : {  
  "Fn::FindInMap" : [   
    "AWSRegionToAMI",  
    { "Ref" : "AWS::Region" },  
    "AMI"  
  ]  
}
```

Use a função intrínseca
Fn::FindInMap para pesquisas

O exemplo aqui mostra uma seção **Mapeamentos** com um mapa chamado *AWSRegionToAMI*. Ele contém quatro chaves que mapeiam para pares nome-valor. Os pares de nome-valor contêm valores de string únicos.

As chaves são nomes de região. Cada par de nome-valor é o ID da AMI na região representada pela chave.

O próximo trecho de JSON mostra como a função intrínseca Fn::FindInMap

O exemplo aqui mostra uma seção **Mapeamentos** com um mapa chamado *AWSRegionToAMI*. Ele contém quatro chaves que mapeiam para pares nome-valor. Os pares de nome-valor contêm valores de string únicos.

As chaves são nomes de região. Cada par de nome-valor é o ID da AMI na região representada pela chave.

O próximo trecho de JSON mostra como a função intrínseca `Fn::FindInMap` pode ser usada para procurar o ID de AMI correto armazenado no mapa. Ele requer o pseudoparâmetro *AWS:Region*, que é resolvido para uma região real.

Para obter mais informações, consulte [Mapeamentos](#).

Definir recursos em um modelo

A seção **Recursos** declara os recursos da AWS a serem criados.

O tipo de recurso que você está declarando.

Você pode exigir a criação de um recurso específico para seguir outro.

Propriedades de um recurso (opcional).

```
"Resources" : {  
  "MyRDSInstance" : {  
    "Type" : "AWS::RDS::DBInstance",  
    "Properties" : {  
      "AllocatedStorage" : "5",  
      ...  
    }  
  },  
  "webServer" : {  
    "Type" : "AWS::EC2::Instance",  
    "DependsOn" : [ "MyRDSInstance" ],  
    "Properties" : {  
      ...  
    }  
  }  
}
```

A seção **Recursos** necessária de um modelo do AWS CloudFormation declara os recursos da AWS que você deseja como parte da sua pilha, como uma instância do Amazon EC2 ou um bucket do Amazon S3. Você deve declarar cada recurso separadamente. No entanto, você pode especificar vários recursos do mesmo tipo. Se você declarar vários recursos, separe-os com vírgulas.

Observe neste exemplo que dois recursos serão criados: uma instância do Amazon Relational Database Service (Amazon RDS) e uma instância do EC2.

A seção **Recursos** necessária de um modelo do AWS CloudFormation declara os recursos da AWS que você deseja como parte da sua pilha, como uma instância do Amazon EC2 ou um bucket do Amazon S3. Você deve declarar cada recurso separadamente. No entanto, você pode especificar vários recursos do mesmo tipo. Se você declarar vários recursos, separe-os com vírgulas.

Observe neste exemplo que dois recursos serão criados: uma instância do Amazon Relational Database Service (Amazon RDS) e uma instância do EC2.

A instância do EC2 — chamada *WebServer* — tem uma dependência da instância do RDS. O banco de dados deve ser criado antes que os aplicativos que serão executados no servidor Web possam usá-lo. Embora o serviço AWS CloudFormation execute um sequenciamento de tarefas de criação de recursos automaticamente, essas regras mudam à medida que o suporte para novos tipos de recursos é implementado por meio do sistema. Para garantir um sequenciamento consistente de tarefas de criação de recursos, é uma prática recomendada usar o atributo *DependsOn*.

Para obter mais informações, consulte [Recursos](#).

CloudFormation::Init

- Tipo de recurso que fornece acesso a metadados de instâncias do EC2.
- Você pode usá-lo com o script auxiliar *cfn-init*.
- **Script auxiliar cfn-init :**
 - Ele lê metadados de modelo da chave `AWS::CloudFormation::Init`.
 - Com essas informações, o *cfn-init* pode realizar as seguintes ações em instâncias do EC2:
 - » Instalar pacotes;
 - » Gerenciar grupos e contas de usuário;
 - » Gravar arquivos no disco;
 - » Executar comandos;
 - » Ativar ou desativar serviços;
 - » Iniciar ou interromper serviços.

No script auxiliar *cfn-init*, use o tipo de recurso `AWS::CloudFormation::Init` para incluir metadados sobre uma instância do EC2. Se o seu modelo chamar o script *cfn-init*, o script procurará metadados de recursos que estão na raiz da chave de metadados `AWS::CloudFormation::Init`. Para obter mais informações sobre o *cfn-init*, consulte [cfn-init](#).

O *cfn-init* oferece suporte a todos os tipos de metadados para sistemas Linux. Ele oferece suporte a tipos de metadados para Windows com condições que são descritas nas seções a seguir. Para obter um exemplo de como usar o

No script auxiliar `cfn-init`, use o tipo de recurso `AWS::CloudFormation::Init` para incluir metadados sobre uma instância do EC2. Se o seu modelo chamar o script `cfn-init`, o script procurará metadados de recursos que estão na raiz da chave de metadados `AWS::CloudFormation::Init`. Para obter mais informações sobre o `cfn-init`, consulte [cfn-init](#).

O `cfn-init` oferece suporte a todos os tipos de metadados para sistemas Linux. Ele oferece suporte a tipos de metadados para Windows com condições que são descritas nas seções a seguir. Para obter um exemplo de como usar o `AWS::CloudFormation::Init` e o script auxiliar `cfn-init`, consulte [Implantação de aplicativos no Amazon EC2 com o AWS CloudFormation](#).

Para obter um exemplo que mostra como usar o `cfn-init` para criar uma pilha do Microsoft Windows, consulte:

- [Inicialização de pilhas do Windows do AWS CloudFormation](#)

Para instâncias do Amazon Elastic Container Service (Amazon ECS), comece com a AMI otimizada para Amazon ECS fornecida pela AWS. Em seguida, usamos o CloudFormation Init para provisionar os pacotes, executar comandos, instalar arquivos e configurar os servidores conforme necessário. A AWS

recomenda esse processo porque ele tem vários benefícios — por exemplo, ele é declarativo e atualizável. `AWS::CloudFormation::Init` é configurado executando scripts `cfn-bootstrap` na instância.

```
"MyInstance": {
  "Tipo": "AWS::EC2::Instance",
  "Metadata" : {
    "AWS::CloudFormation::Init" : {
      "webapp-config": {
        "packages" : {},
        "sources"   : {}, "files"   : {},
        "groups"   : {}, "users"   : {},
        "commands" : {}, "services" : {}
      }
    }
  }
}
```

Para obter mais informações, consulte a página [AWS::CloudFormation::Init](#).

Dados do usuário em comparação ao CloudFormation::Init

Dados do usuário no AWS CloudFormation:

- Arquivo de script, como acontece com instâncias individuais.
- Oferece maior controle.
- Oferece maior potencial de erro. Deve ser mais cauteloso.

CloudFormation::Init:

- Pode reverter automaticamente em caso de falha.
- Mais fácil de gerenciar do que campos de dados do usuário em modelos do CloudFormation.
- Pode anexar metadados dentro do modelo.
- Conjuntos de configurações permitem várias configurações em um único CloudFormation::Init.

Para configurar uma instância, você deve optar por usar CloudFormation::Init ou dados do usuário.

Os dados do usuário oferecem maior controle sobre a ordem de execução das suas operações. Ele também oferece a possibilidade de uma manipulação de erros e recuperação de erros mais robustos, pois os scripts de dados do usuário são escritos em linguagens de programação com loops e suporte para condicionais.

Uma desvantagem do uso de dados de usuário em um modelo do AWS

Para configurar uma instância, você deve optar por usar `CloudFormation::Init` ou dados do usuário.

Os dados do usuário oferecem maior controle sobre a ordem de execução das suas operações. Ele também oferece a possibilidade de uma manipulação de erros e recuperação de erros mais robustos, pois os scripts de dados do usuário são escritos em linguagens de programação com loops e suporte para condicionais.

Uma desvantagem do uso de dados de usuário em um modelo do AWS CloudFormation é que qualquer caractere especial, como aspas duplas, deve ser recuado dentro do modelo para que o arquivo seja analisado como código JSON ou YAML válido. Pode ser complicado escapar dos dados do usuário para que eles sejam analisados corretamente como código do JSON ou do YAML.

Uma estratégia é colocar todo o código de dados do usuário em um script separado que o AWS CloudFormation executa com um `CloudFormation::Init`. Essa solução é mais limpa e fácil de manter do que incorporar código de programação nos seus modelos. Para facilitar o desenvolvimento e a manutenção, é uma boa ideia testar métodos diferentes de inicialização de instâncias do EC2 por meio do AWS CloudFormation e padronizar um método uniforme usado em toda a organização.

Você pode criar vários conjuntos de configurações e chamar uma série deles usando o script `cfn-init`. Cada configset pode conter uma lista de chaves de configuração ou referências a outros configsets. Conjuntos de configurações são usados com os comandos `cf-init` para permitir a reutilização mais fácil da reutilização de código. Por exemplo, você pode definir um conjunto de configurações padrão para todos os servidores Web para instalar o Apache e o PHP, em vez de criar os scripts de dados do usuário para instalá-los a cada vez ou construí-lo em uma AML.

WaitCondition e WaitConditionHandle

O `WaitCondition` e o `WaitConditionHandle` são usados quando você inicializa instâncias. Além disso:

- O `WaitCondition` bloqueia o estado de conclusão de uma pilha até que `WaitConditionHandle` seja chamado ou o limite de tempo limite seja atingido.
 - Você pode especificar o número de sinais bem-sucedidos que devem ser recebidos antes que o `WaitCondition` seja atendido.
 - Padrão: 1.
- Você pode sinalizar sucesso ou falha usando o comando `cfn-signal`

Para a AWS, uma instância do EC2 está pronta quando passa nas verificações de status. No entanto, passar nas verificações de status pode não significar que a sua instância esteja pronta em termos das instalações e das configurações que você espera que a instância do EC2 tenha. Os dados e scripts do usuário ainda podem estar instalando um software que transforma sua instância em um servidor de banco de dados, um servidor do SharePoint ou um nó em um cluster de processamento de dados.

Para sinalizar ao AWS CloudFormation que a sua instância está realmente pronta, use uma *WaitCondition*. Considere uma *WaitCondition* como um sinal de interrupção que impede que a pilha seja marcada como concluída pelo AWS

Para a AWS, uma instância do EC2 está pronta quando passa nas verificações de status. No entanto, passar nas verificações de status pode não significar que a sua instância esteja pronta em termos das instalações e das configurações que você espera que a instância do EC2 tenha. Os dados e scripts do usuário ainda podem estar instalando um software que transforma sua instância em um servidor de banco de dados, um servidor do SharePoint ou um nó em um cluster de processamento de dados.

Para sinalizar ao AWS CloudFormation que a sua instância está realmente pronta, use uma *WaitCondition*. Considere uma *WaitCondition* como um sinal de interrupção que impede que a pilha seja marcada como concluída pelo AWS CloudFormation antes que o código envie um sinal de que ela foi concluída. *WaitConditionHandle* é uma representação textual do *WaitCondition* que o código de inicialização pode usar para sinalizar ao AWS CloudFormation que o recurso foi criado.

Seu código de inicialização é responsável por chamar o comando *cfn-signal* na instância para indicar que ela terminou o trabalho. Isso é verdade se o código está nos dados do usuário ou em um script separado que é baixado para a instância usando *CloudFormation::Init*. Você pode sinalizar que a tarefa foi bem-sucedida. Se uma tarefa for bem-sucedida, o AWS CloudFormation começará a

criar o próximo recurso na sua pilha ou marcará toda a pilha como concluída. Ou você pode marcar que a tarefa falhou. Se uma tarefa falhar, por padrão, o AWS CloudFormation reverterá todos os recursos na sua pilha que foram criados até o momento.

WaitCondition e WaitConditionHandle

Especificar um WaitConditionHandle

```
"WebServerWaitHandle": {
  "Type":
    "AWS::CloudFormation::WaitConditionHandle"
},
"WebServerWaitCondition": {
  "Type":
    "AWS::CloudFormation::WaitCondition",
  "DependsOn": "WebServerInstance",
  "Properties": {
    "Handle": {
      "Ref": "WebServerWaitHandle"
    },
    "Timeout": "3600"
  }
}
```

Sinalização de um WaitCondition

```
"UserData": {
  "Fn::Base64": {
    "Fn::Join": [ "", [
      "#!/bin/bash\n",
      "/opt/aws/bin/cfn-signal -e 0\n",
      "-r \"complete\" \"\"",
      {
        "Ref":
          "WebServerWaitHandle"
      }, "\""\n"
    ] ]
  ]
}
```

A seção de exemplo de um modelo do AWS CloudFormation à esquerda define um recurso `AWS::CloudFormation::WaitConditionHandle` chamado `WebServerWaitHandle`.

Mais abaixo, o mesmo snippet JSON define um `AWS::CloudFormation::WaitCondition`, que é chamado `WebServerWaitCondition`. Você pode ver que ele aguarda para receber a entrada neste `WaitConditionHandle`.

A seção de exemplo de um modelo do AWS CloudFormation à esquerda define um recurso `AWS::CloudFormation::WaitConditionHandle` chamado *WebServerWaitHandle*.

Mais abaixo, o mesmo snippet JSON define um `AWS::CloudFormation::WaitCondition`, que é chamado *WebServerWaitCondition*. Você pode ver que ele aguarda para receber a entrada neste `WaitConditionHandle`.

O bloco de código à direita é um exemplo da seção **UserData** de um recurso `AWS::EC2::Instance`. Ele invoca o comando `cfn-signal` e tem uma referência ao `WebServerWaitHandle`. Depois que a execução do script `UserData` for concluída, o `WaitConditionHandle` será atendido.

Define saídas em um modelo

Saídas:

- Valores de retorno para recursos criados como parte da pilha.
- Use funções intrínsecas para obter valores de recursos na pilha.

```
"Outputs" : {  
  "WebsiteURL" : {  
    "Description" : "The URL of the website",  
    "Value" : { "Fn::Join" : [ "", [ "http://", {  
      "Fn::GetAtt" : [ "ElasticLoadBalancer", "DNSName" ] } ] ] }  
  }  
}
```

Saídas é uma seção *opcional* em um modelo do AWS CloudFormation. A seção **Saídas** retorna valores de string que foram criados pelo modelo e que podem ser importantes para os usuários. Por exemplo, se você criar uma instância do EC2 que funcione como um servidor Web público, poderá optar por produzir o nome DNS (Domain Name Server) público completo do servidor na seção **Saídas**.

No exemplo, o nome DNS de um balanceador de carga do Elastic Load Balancing — que também deve ser definido na seção **Recursos** do mesmo modelo do AWS CloudFormation — será retornado. A URL do site que está disponível por meio do balanceador de carga será conhecida.

Depois de executar a pilha, as saídas ficam visíveis na guia **Saídas** do console do

Saídas é uma seção *opcional* em um modelo do AWS CloudFormation. A seção **Saídas** retorna valores de string que foram criados pelo modelo e que podem ser importantes para os usuários. Por exemplo, se você criar uma instância do EC2 que funcione como um servidor Web público, poderá optar por produzir o nome DNS (Domain Name Server) público completo do servidor na seção **Saídas**.

No exemplo, o nome DNS de um balanceador de carga do Elastic Load Balancing — que também deve ser definido na seção **Recursos** do mesmo modelo do AWS CloudFormation — será retornado. A URL do site que está disponível por meio do balanceador de carga será conhecida.

Depois de executar a pilha, as saídas ficam visíveis na guia **Saídas** do console do AWS CloudFormation. Se você executar sua pilha usando a AWS CLI ou a API, também poderá recuperar as saídas programaticamente.

Opções adicionais de pilha do AWS CloudFormation

Opções adicionais podem ser especificadas ao criar uma pilha do AWS CloudFormation, como:

- Como evitar a *reversão em caso de falha*.
- Definir uma *política de pilha* para controlar atualizações de pilha.
- Habilitar proteção contra encerramento.

```
aws cloudformation create-stack --stack-name NewStack  
--template-body file://path/to/template.yml --on-failure DO_NOTHING
```

Embora o comportamento padrão do AWS CloudFormation seja *reverter* todas as alterações quando uma pilha não concluir todas as ações com êxito, você pode substituir o padrão.

Você também pode aplicar uma *política de pilha* para impedir atualizações nos recursos da pilha. Quando você cria uma pilha, todas as ações de atualização são permitidas em todos os recursos, por padrão. Isso significa que qualquer pessoa com permissões de atualização da pilha pode atualizar todos os recursos na pilha. Você pode impedir que os recursos da pilha sejam

Embora o comportamento padrão do AWS CloudFormation seja *reverter* todas as alterações quando uma pilha não concluir todas as ações com êxito, você pode substituir o padrão.

Você também pode aplicar uma *política de pilha* para impedir atualizações nos recursos da pilha. Quando você cria uma pilha, todas as ações de atualização são permitidas em todos os recursos, por padrão. Isso significa que qualquer pessoa com permissões de atualização da pilha pode atualizar todos os recursos na pilha. Você pode impedir que os recursos de pilha sejam acidentalmente atualizados ou excluídos durante uma atualização de pilha usando uma *política de pilha*.

A política determina qual ação executar se a criação da pilha falhar, que deve ser uma das seguintes: *DO_NOTHING*, *ROLLBACK* ou *DELETE*. Você pode especificar *on-failure* ou *disable-rollback*, mas não ambos. No exemplo de comando da AWS CLI, você pode ver que o comando `create-stack` está sendo executado e que as opções de reversão padrão em caso de falha foram substituídas. Em vez de reverter em caso de falha, o AWS CloudFormation não fará nada, o que significa que todos os recursos criados pela pilha com falha serão deixados inalterados pelo AWS CloudFormation.

Você também pode impedir que uma pilha seja excluída acidentalmente

habilitando a proteção contra encerramento na pilha. Se um usuário tentar excluir uma pilha e a proteção contra encerramento estiver habilitada, a exclusão falhará e a pilha, incluindo o seu status, permanecerá inalterada.

Substituição de reversões de atualização com falha

- Continue revertendo uma atualização para a sua pilha, mesmo após a reversão ter falhado e estar no estado `UPDATE_ROLLBACK_FAILED`
- Execute uma substituição:
 - Corrija a causa da falha na reversão.
 - Instruir o AWS CloudFormation a continuar revertendo a atualização.

```
aws cloudformation continue-update-rollback --stack-name ExistingStack
```

Você pode instruir o AWS CloudFormation a continuar revertendo uma atualização para a sua pilha mesmo depois que a reversão tiver falhado. Algumas causas comuns de uma reversão com falha incluem alterar um recurso em sua pilha fora do AWS CloudFormation, permissões insuficientes, erros de limitação ou recursos que foram estabilizados.

Uma pilha entra no estado `UPDATE_ROLLBACK_FAILED` quando o AWS CloudFormation não pode reverter todas as alterações durante uma atualização. Por exemplo, você pode ter uma pilha que está sendo revertida para uma

Você pode instruir o AWS CloudFormation a continuar revertendo uma atualização para a sua pilha mesmo depois que a reversão tiver falhado. Algumas causas comuns de uma reversão com falha incluem alterar um recurso em sua pilha fora do AWS CloudFormation, permissões insuficientes, erros de limitação ou recursos que foram estabilizados.

Uma pilha entra no estado *UPDATE_ROLLBACK_FAILED* quando o AWS CloudFormation não pode reverter todas as alterações durante uma atualização. Por exemplo, você pode ter uma pilha que está sendo revertida para uma instância de banco de dados antiga que foi excluída fora do AWS CloudFormation.

Depois de corrigir a causa da falha na reversão, você pode instruir o AWS CloudFormation a continuar a reverter a atualização usando o console do AWS CloudFormation ou a AWS CLI.

Para obter mais informações, consulte [continue-update-rollback](#) na AWS CLI

Conjuntos de alterações

- Os conjuntos de alterações mostram as alterações propostas em uma pilha antes de qualquer implementação.
- O AWS CloudFormation compara as alterações enviadas à pilha.
- Os engenheiros podem visualizar as alterações para ver quais recursos serão adicionados, modificados ou excluídos.
- O conjunto de alterações pode então ser aplicado à pilha para implementar as alterações.



26

Os conjuntos de alterações permitem que os engenheiros entendam exatamente o que acontecerá com uma alteração em uma pilha antes da implementação. Isso garante que todas e quaisquer atualizações, alterações e exclusões serão identificadas antes da implementação.

As etapas na criação de um conjunto de alterações são:

1. Criar alterações para a pilha de destino. Em seguida, o AWS CloudFormation cria um conjunto de alterações.
2. Revisar o conjunto de alterações.
3. Se necessário, faça as alterações adicionais necessárias. Para cada alteração

Os conjuntos de alterações permitem que os engenheiros entendam exatamente o que acontecerá com uma alteração em uma pilha antes da implementação. Isso garante que todas e quaisquer atualizações, alterações e exclusões serão identificadas antes da implementação.

As etapas na criação de um conjunto de alterações são:

1. Criar alterações para a pilha de destino. Em seguida, o AWS CloudFormation cria um conjunto de alterações.
2. Revisar o conjunto de alterações.
3. Se necessário, faça as alterações adicionais necessárias. Para cada alteração adicional, repeta as etapas 1 e 2.
4. Executar o conjunto de alterações para implementar as alterações.

Práticas recomendadas do AWS CloudFormation (1 de 3)

Planejamento e organização:

Organize as suas pilhas por ciclo de vida e propriedade.

Reutilize modelos para replicar pilhas em vários ambientes.

Exemplo: ambientes de desenvolvimento, teste e produção.

Verifique limites para todos os tipos de recursos.

27



As práticas recomendadas apresentadas aqui são recomendações que podem ajudá-lo a usar o AWS CloudFormation de forma mais eficaz e segura.

No que diz respeito ao planejamento e à organização do uso do AWS CloudFormation:

- Use o ciclo de vida e a propriedade de seus recursos da AWS para ajudá-lo a decidir quais recursos devem ser colocados em cada pilha. À medida que sua pilha cresce em escala e se expande no escopo, gerenciar uma única pilha pode ser complicado e demorado. Ao agrupar recursos com ciclos de vida e propriedade comuns, os proprietários podem fazer alterações no conjunto



As práticas recomendadas apresentadas aqui são recomendações que podem ajudá-lo a usar o AWS CloudFormation de forma mais eficaz e segura.

No que diz respeito ao planejamento e à organização do uso do AWS CloudFormation:

- Use o ciclo de vida e a propriedade de seus recursos da AWS para ajudá-lo a decidir quais recursos devem ser colocados em cada pilha. À medida que sua pilha cresce em escala e se expande no escopo, gerenciar uma única pilha pode ser complicado e demorado. Ao agrupar recursos com ciclos de vida e propriedade comuns, os proprietários podem fazer alterações no conjunto de recursos deles usando um processo e uma programação próprios sem afetar outros recursos.
- Reutilize modelos para replicar pilhas em vários ambientes. Depois de configurar suas pilhas e recursos, você pode reutilizar os modelos para replicar a sua infraestrutura em vários ambientes. Por exemplo, você pode criar ambientes para desenvolvimento, teste e produção, para que possa testar alterações antes de implementá-las na produção.
- Verifique cotas para todos os tipos de recursos. Antes de iniciar uma pilha, certifique-se de que você possa criar todos os recursos desejados sem

exceder os limites da conta da AWS.

- As práticas recomendadas do AWS CloudFormation oferecem uma infraestrutura repetível para implantação fácil. Embora você possa implantar a infraestrutura sem intervenção, é importante observar os limites de serviço do Amazon EC2 e outros limites de serviço da AWS que podem afetar a implantação dos seus scripts do AWS CloudFormation. Você pode usar as verificações do AWS Trusted Advisor para garantir que os limites de determinadas instâncias do EC2 não serão ultrapassados e, se esse for o caso, solicitar um aumento do limite de serviço da ferramenta.
- Use o AWS Organizations em conjunto com o AWS CloudFormation pode ajudar a garantir que a infraestrutura criada em contas separadas siga as mesmas políticas e controles organizacionais do AWS Identity and Access Management (IAM).

Práticas recomendadas do AWS CloudFormation (2 de 3)

Criação de modelos:

Não incorpore credenciais em seus modelos.

Use tipos de parâmetros específicos da AWS.

Use restrições de parâmetros.

Use o `AWS::CloudFormation::Init` para implantar aplicativos de software em instâncias do Amazon EC2.

Valide modelos antes de usá-los.

Veja a seguir algumas práticas recomendadas relacionadas à criação de modelos do AWS CloudFormation:

- Evite incorporar credenciais em seus modelos. Em vez disso, use *parâmetros de entrada* para passar informações ao criar ou atualizar uma pilha.
- Use tipos de parâmetros específicos da AWS. O AWS CloudFormation pode

Veja a seguir algumas práticas recomendadas relacionadas à criação de modelos do AWS CloudFormation:

- Evite incorporar credenciais em seus modelos. Em vez disso, use *parâmetros de entrada* para passar informações ao criar ou atualizar uma pilha.
- Use tipos de parâmetros específicos da AWS. O AWS CloudFormation pode validar rapidamente valores para tipos de parâmetros específicos da AWS antes de criar a pilha.
- Use restrições de parâmetros. Com restrições, você pode descrever valores de entrada permitidos para que o AWS CloudFormation detecte todos os valores inválidos antes de criar uma pilha.
- Use o `AWS::CloudFormation::Init` para implantar aplicativos de software em instâncias do Amazon EC2. Ao usar o `AWS::CloudFormation::Init`, você pode descrever as configurações desejadas, em vez de criar scripts de etapas processuais.
- Valide modelos antes de usá-los. A validação de um modelo pode ajudar você a detectar erros de sintaxe e alguns erros semânticos antes que o AWS CloudFormation crie qualquer recurso.

Gerenciamento de pilhas:

Gerencie todos os recursos de pilha por meio do AWS CloudFormation.

Crie conjuntos de alterações antes de atualizar suas pilhas.

Use políticas de pilha.

Use o AWS CloudTrail para registrar chamadas do AWS CloudFormation.

Use revisões de código e controles de revisão para gerenciar os seus modelos.

Por fim, em relação ao gerenciamento das suas pilhas, aqui estão algumas práticas recomendadas adicionais:

- Gerencie todos os recursos de pilha por meio do AWS CloudFormation. Depois de iniciar uma pilha, use o console do AWS CloudFormation, a API ou a AWS CLI para atualizar os recursos em sua pilha. Não faça alterações em recursos de pilha fora do AWS CloudFormation. Isso pode criar uma incompatibilidade entre o modelo da pilha e o estado atual dos recursos da pilha, o que pode causar erros se você atualizar ou excluir a pilha.
- Crie *conjuntos de alterações* antes de atualizar as pilhas. Os conjuntos de alterações permitem que você veja como as alterações propostas para uma

Por fim, em relação ao gerenciamento das suas pilhas, aqui estão algumas práticas recomendadas adicionais:

- Gerencie todos os recursos de pilha por meio do AWS CloudFormation. Depois de iniciar uma pilha, use o console do AWS CloudFormation, a API ou a AWS CLI para atualizar os recursos em sua pilha. Não faça alterações em recursos de pilha fora do AWS CloudFormation. Isso pode criar uma incompatibilidade entre o modelo da pilha e o estado atual dos recursos da pilha, o que pode causar erros se você atualizar ou excluir a pilha.
- Crie *conjuntos de alterações* antes de atualizar as pilhas. Os conjuntos de alterações permitem que você veja como as alterações propostas para uma pilha podem afetar os seus recursos em execução antes de implementá-las. O AWS CloudFormation não faz alterações na pilha até que você execute o conjunto de alterações, o que permite que você decida se deseja prosseguir com as alterações propostas ou criar outro conjunto de alterações.
- Use políticas de pilha. As políticas de pilha ajudam a proteger recursos de pilha críticos contra atualizações não intencionais que podem fazer com que os recursos sejam interrompidos ou até mesmo substituídos. Especifique uma política de pilha ao criar uma pilha que tenha recursos críticos.

- Use o AWS CloudTrail para registrar chamadas do AWS CloudFormation. O AWS CloudTrail rastreia qualquer pessoa que faz chamadas de API do AWS CloudFormation na sua conta da AWS. As chamadas de API são registradas em log quando qualquer pessoa usa a API do AWS CloudFormation, o console do AWS CloudFormation, um console de back-end ou os comandos de AWS CLI para o AWS CloudFormation. Habilite o registro em log e especifique um bucket do S3 para armazenar os logs. Ao fazer isso, você pode auditar quem fez qual chamada do AWS CloudFormation na sua conta quando necessário.
- Use revisões de código e controles de revisão para gerenciar modelos. Os modelos de pilha descrevem a configuração de seus recursos da AWS, como os valores das propriedades deles. Esses métodos ajudam a rastrear alterações entre diferentes versões de modelos, o que ajuda a rastrear alterações em recursos de pilha. Além disso, ao manter um histórico, você pode reverter a pilha para uma determinada versão do modelo.

Principais conclusões



© 2020, Amazon Web Services, Inc. ou suas afiliadas. Todos os direitos reservados.

30

- O AWS CloudFormation permite criar e provisionar implantações de infraestrutura da AWS de forma previsível e repetida.
- Dois termos principais para o AWS CloudFormation são *modelos* e *pilhas*.
- Se um erro for encontrado quando você iniciar um modelo do AWS CloudFormation, todos os recursos serão revertidos por padrão.
- Os parâmetros permitem que você insira valores personalizados em seu modelo toda vez que criar ou atualizar uma pilha.



- O AWS CloudFormation permite criar e provisionar implantações de infraestrutura da AWS de forma previsível e repetida.
- Dois termos principais para o AWS CloudFormation são *modelos* e *pilhas*.
- Se um erro for encontrado quando você iniciar um modelo do AWS CloudFormation, todos os recursos serão revertidos por padrão.
- Os parâmetros permitem que você insira valores personalizados em seu